

Pre-Lab Report

- Lab 07 -

< 제목 >

담당교수	박병은
담당조교	심범식
실험일자	26. 9. 8.
학과	전자전기컴퓨터공학부
학번	2023440075
이름	유도현

목차

1. 초록 (Abstract)	-----
2. 소개 (Introduction)	-----
1) 실험 목적	
3. 예비 보고서	-----
4. 참고문헌	-----

1. 초록 (Abstract)

현재 우리는 여러 전자기기를 일상에서 사용하고 있다. 이러한 전자기기 내에는 메모리, 마이크로프로세서 등 논리회로를 이용한 기기들이 탑재되어 있다. 오늘날 엔지니어들은 이러한 회로를 구현하는데 HDL이라는 특수한 언어를 사용한다. HDL은 디지털 시스템을 텍스트 형태로 기술하는 언어로 logic diagram, truth table, Boolean expression, abstraction of digital system 등을 기술하는데 이용될 수 있다.[1] 본 실험에서는 AMD Vivado Design Suite를 이용해 HDL의 한 종류인 Verilog를 사용해보고, 입력에 따른 LED 점등 로직을 구현할 것이다. 또한 구현한 로직을 FPGA에 프로그래밍 하여 구현한 로직이 물리적으로 어떻게 실현되는지 관찰할 것이다.

2. 소개 (Introduction)

1. HDL 소개

HDL은 텍스트 기반의 하드웨어 기술언어로 복잡한 회로 설계 시 오류 발생 가능성을 축소시키고 비용을 줄일 수 있다는 장점이 있다. 또한 HDL을 이용해 디지털 시스템을 사람과 컴퓨터 둘 모두에게 읽힐 수 있는 문서로 기술하고 컴퓨터 소프트웨어를 통해 Logic simulation, Logic synthesis, Time verification 등을 수행할 수 있다. 디지털 시스템을 설계할 때 HDL은 아래와 같은 주요한 몇 단계에서 사용된다. [1]

A. Design entry

Design entry에서는 하드웨어에서 실행될 기능을 HDL로 기술한다. 하드웨어에서 실행될 기능을 기술한 HDL 문서는 Boolean logic equation, truth table, netlist of interconnected gate, abstract behavioral model 등 다양한 방식을 통해 표현될 수 있다. [1]

B. Logic simulation

Logic simulation은 컴퓨터를 통해 디지털 시스템이 어떻게 작동하는지 보여준다. 시뮬레이터는 HDL을 읽어 input, output 관계를 시간에 따른 파형 등을 통해 보여준다. 엔지니어는 이 과정을 통해 기능적인 결함을 물리적인 기기를 만들지 않고 잡아낼 수 있다. [1]

C. Logic synthesis

Logic synthesis는 정형화된 과정을 통해 digital system을 구성하는 요소들의 수를 최소화한다. 이 과정을 통해 최소화, 최적화된 요소들의 연결 목록을 얻을 수 있고 이를 netlist라 한다. [1]

D. Timing verification

Timing verification 과정에서는 synthesized, fabricated된 디지털 시스템이 지정한 속도로 움직이는지 확인한다. 디지털 시스템은 논리 게이트로 구성되어 있고 각 게이트는 물리적 요소에 의해 지연을 내포하고 있다. 따라서 본 과정을 통해 디지털 시스템의 지연이 타협 가능한 정도인지 확인한다. [1]

E. Fault simulation

디지털 시스템이 만들어지는 과정에서, Clean room 내부의 미세 먼지, 입자 등이 디지털 시스템에 결함을 야기할 수 있다. Fault simulation 과정에서는 실제로 만들어진 디지털 시스템의 작동을 이상적 디지털 시스템의 작동과 비교하여 만들어진 디지털 시스템이 정상 작동하는지 확인한다. [1]

1) 실험목적

3. 예비 보고서

- 2-bit AND, OR, XOR, NOR, NAND의 진리표

gate 입력		AND	OR	XOR	NOR	NAND
x	Y					
0	0	0	0	0	1	1
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0

- AND, OR, XOR, NOR, NAND의 Verilog 연산자 표현

Gate	Operator(Bitwise)
AND	&
OR	
XOR	^
NOR	~
NAND	~&

- 2개의 input을 갖고 5개의 output이 각각 AND, OR, XOR, NOR, NAND 게이트인 logic의 Verilog 코드. Input과 output은 아래 표를 참조해 DIP switch와 LED에 연결

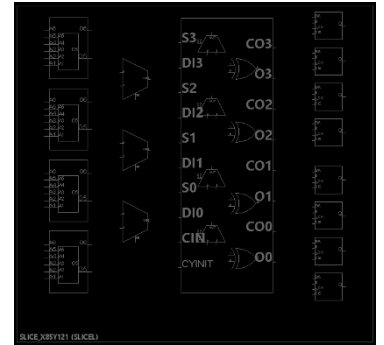
- github 주소:

Signal name	Pin Number
DIP1	Y1
DIP2	W3
DIP3	U2

Signal name	Pin Number
DIP1	Y1
DIP2	W3
DIP3	U2

■ 위 Logic에서 FPGA상에서 예상되는 SLICE, LUT의 사용량

- ◆ AND, OR, XOR, NOR, NAND는 각각 LUT2로 구현 가능하다. 그러므로 총 LUT 10개가 필요할 것으로 예상된다. 하나의 SLICE에 옆의 사진과 같이 4개의 LUT6가 있고 하나의 LUT6는 5bit 이하의 LUT로 사용할 경우 이를 나눠서 두 개의 LUT로 사용할 수 있으므로 LUT2 4개를 2개의 LUT6에 매핑하고 남은 하나의 LUT2 또한 남은 2개 중 하나의 LUT6에 매핑될 것으로 예상된다. 그러므로 총 하나의 SLICE와 3개의 LUT를 사용할 것으로 예상된다.



■ 위 Logic을 implementation까지 수행 후 나온 결과 레포트에 나온 slice와 LUT 사용 개수

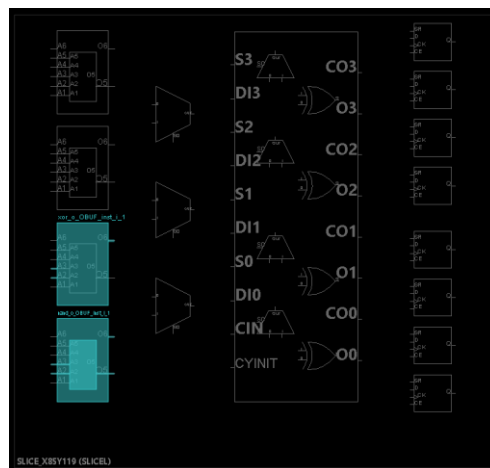
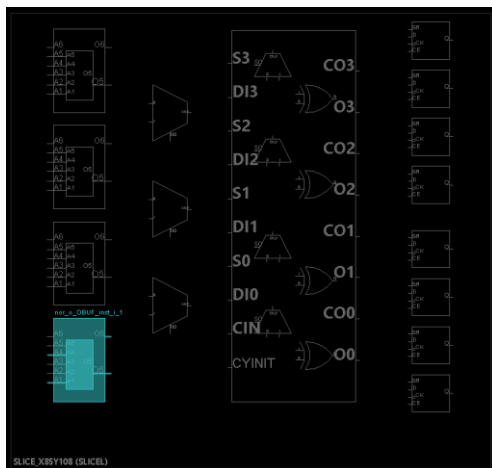
- ◆ LUT6은 예상대로 3개가 사용되었으나 한 SLICE 내의 LUT6을 모두 사용하는 것이 아닌 2개의 SLICE에서 각각 LUT6을 1개, 2개로 사용된 것을 확인하였다. 물리적 회로에서 발생하는 지연을 최소화하기 위해 최적화 과정에서 이와 같이 배치된 것으로 예상된다.

1. Slice Logic

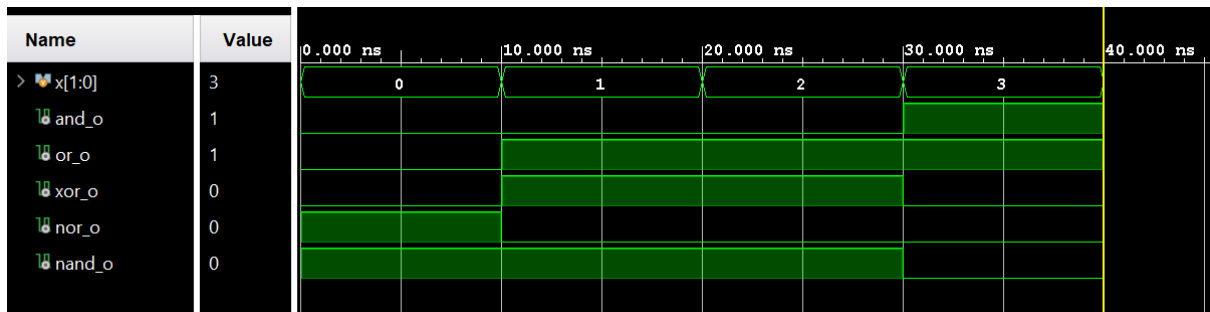
Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs	3	0	0	48000	<0.01
LUT as Logic	3	0	0	48000	<0.01
LUT as Memory	0	0	0	17600	0.00
Slice Registers	0	0	0	96000	0.00
Register as Flip Flop	0	0	0	96000	0.00
Register as Latch	0	0	0	96000	0.00
F7 Muxes	0	0	0	32000	0.00
F8 Muxes	0	0	0	16000	0.00
Unique Control Sets	0	0	0	16000	0.00

2. Slice Logic Distribution

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice	2	0	0	16000	0.01
SLICE1	2	0	0	16000	0.01
SLICE2	0	0	0	16000	0.00
LUT as Logic	3	0	0	48000	<0.01
using O5 output only	0	0	0	17600	0.00
using O6 output only	1	0	0	17600	0.00
using O5 and O6	2	0	0	17600	0.00
LUT as Memory	0	0	0	17600	0.00
LUT as Distributed RAM	0	0	0	17600	0.00
using O5 output only	0	0	0	17600	0.00
using O6 output only	0	0	0	17600	0.00
using O5 and O6	0	0	0	17600	0.00
LUT as Shift Register	0	0	0	17600	0.00
using O5 output only	0	0	0	17600	0.00
using O6 output only	0	0	0	17600	0.00
using O5 and O6	0	0	0	17600	0.00
Slice Registers	0	0	0	96000	0.00
Register driven from within the Slice	0	0	0	96000	0.00
Register driven from outside the Slice	0	0	0	96000	0.00



- 시뮬레이션으로 input을 2'b00부터 2'b11까지 변화시켰을 때, output 파형



◆ 본 시뮬레이션에서 AND 게이트의 결과 진리표

- 본 섹션에서 작성한 진리표와 시뮬레이션이 보여주는 결과값이 동일한 것을 확인할 수 있다.

Input: x[1:0]	Output(AND)
00	0
01	0
10	0
11	1

- Half adder의 진리표 및 카르노 맵을 통해 구한 Carry와 Sum에 대한 논리식

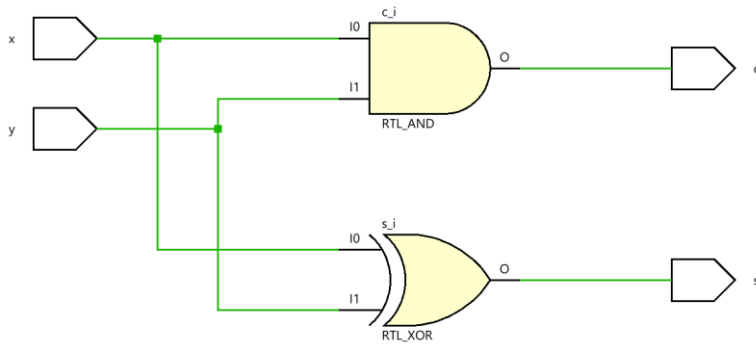
Input		Output(Half Adder)	
X	y	Sum	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Sum			Carry		
x \ y	0	1	x \ y	0	1
0			0		
1	1	1	1		1
Sum = $xy' + x'y$			Carry = xy		

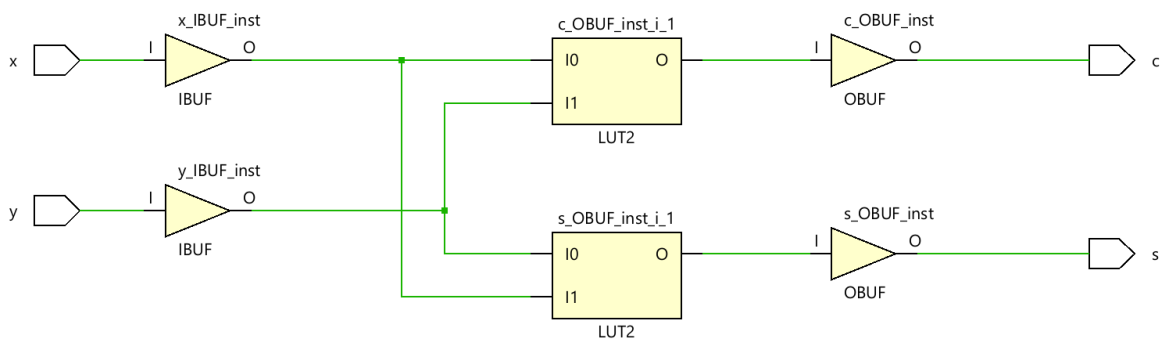
- Half adder를 위에서 구한 논리식을 이용해 Verilog 코드를 이용하여 작성 및 구현

◆ 코드: github 참고

◆ Synthesis 하기 전 schematic



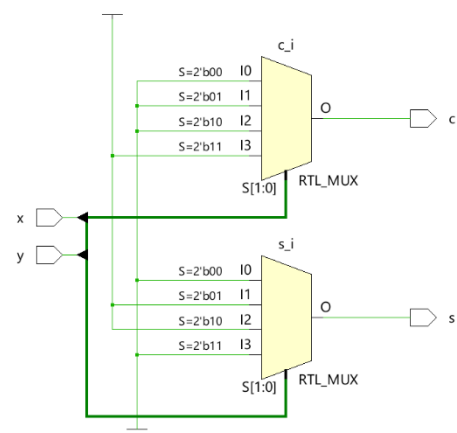
◆ Implementation 한 후 schematic



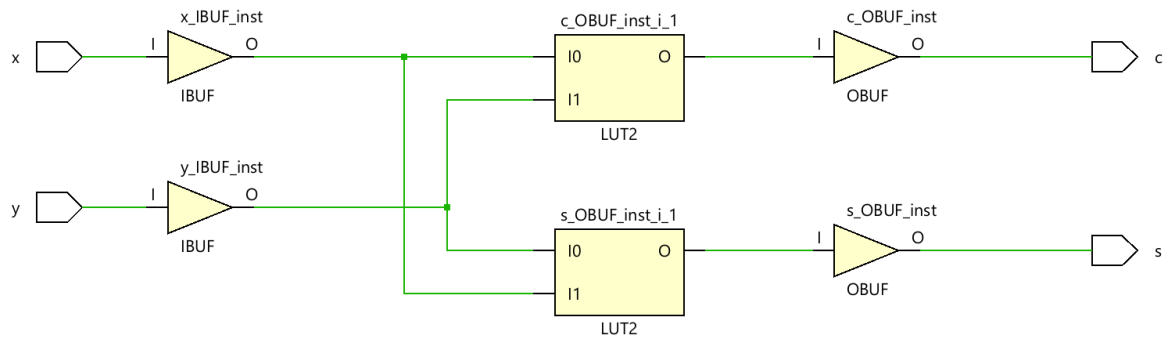
- Half adder를 case문을 이용해 Verilog 코드로 작성

◆ 코드: github 참고

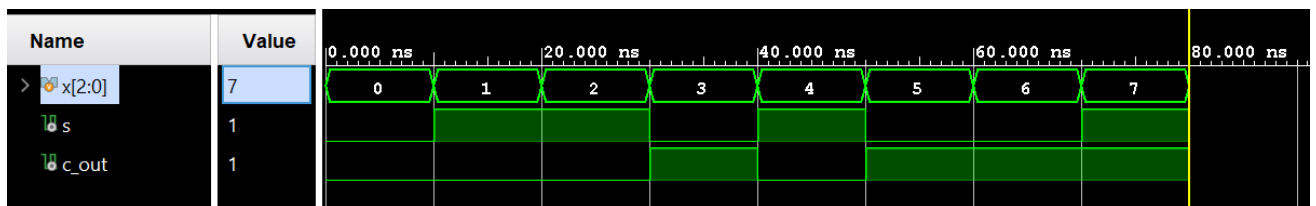
◆ Synthesis 하기 전 schematic



◆ Implementation 한 후 schematic



- Synthesis 하기 전 논리식을 이용한 schematic은 AND, XOR gate로 구성되는데 반해 case를 이용한 schematic은 multiplexer를 이용한 것을 확인할 수 있다.
 - Implementation 후 논리식, case 구문을 이용한 schematic들의 차이가 없는 것을 확인 가능하다.
- Full adder 구현 및 input을 3'b000~3'b111까지 바꿨을 때 input/output 시뮬레이션 파형
 - 코드: [github](#) 참고
 - input/output 시뮬레이션 파형



4. 참고문헌

[1] digital design